Software Architecture and Design Specification

Project: Online Exam Portal

Team Number: 9

Version: 1.0

Authors: Aadish Sarin (Lead), A S Vincent, Aditya Rao, Akshansh Agrawal

Date: 14-09-2026

Status: Final

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Team 9	Initial completed Software Architecture and Design Specification based on the final SRS.

Approvals

Role	Name	Signature/Date
Course Coordinator		
Team Lead	Aadish Sarin (PES2UG24CS007)	

1. Introduction

1.1 Purpose

This document specifies the proposed software architecture and detailed design for the Online Exam Portal. It translates the functional, non-functional, and security requirements in the final SRS into components, interfaces, data responsibilities, interaction flows, security controls, and design decisions.

1.2 Scope

The design covers user registration and authentication, role-based access for Student, Instructor and Admin, exam and question-bank management, timed exam delivery, auto-save and auto-submit, automatic and manual evaluation, result publication, and instructor analytics. The SRS explicitly excludes payment/billing, proctoring hardware, and external LMS integration beyond documented APIs.

1.3 Audience

Developers, QA engineers, instructors, students, system administrators, security reviewers, and assessment evaluators.

1.4 Definitions

Term	Meaning
OEP	Online Exam Portal
MCQ	Multiple Choice Question
RBAC	Role-Based Access Control
OTP	One-Time Password
UI	User Interface
API	Application Programming Interface
TLS	Transport Layer Security
RTM	Requirements Traceability Matrix
WCAG	Web Content Accessibility Guidelines
SAD	Software Architecture and Design Specification

1.5 Design Basis

The design is derived from the final SRS dated 09-09-2026. The SRS defines a browser-based front end, an application/API server, a relational database, HTTPS communication, and optional email/SMS gateway integration. It also requires measurable performance, scalability, reliability, accessibility, and security controls.

2. Document Overview

2.1 How to use this document

Sections 3 and 4 describe the architecture and detailed design. The architecture section identifies major components, responsibilities, interfaces, technology assumptions, risks, traceability, and security. The design section describes key interaction flows, API contracts, error handling, logging, monitoring, and user-experience considerations.

2.2 Related Documents

- Software Requirements Specification (SRS), Online Exam Portal, Team 9, Version 1.1, dated 09-09-2026.
- Requirements Traceability Matrix contained in the final SRS.
- UML use-case diagrams contained in the final SRS.

2.3 Design Principles

- Separate presentation, application/service, persistence, and integration responsibilities.
- Enforce authorization on the server rather than relying only on the browser UI.
- Keep correct answers inaccessible to students during an active exam.
- Persist answers periodically so a client disconnect does not lose in-progress work.
- Use consistent API contracts and standardized errors.
- Keep audit information sufficient to trace authentication, submission, and grading actions without logging sensitive credentials.

3. Architecture

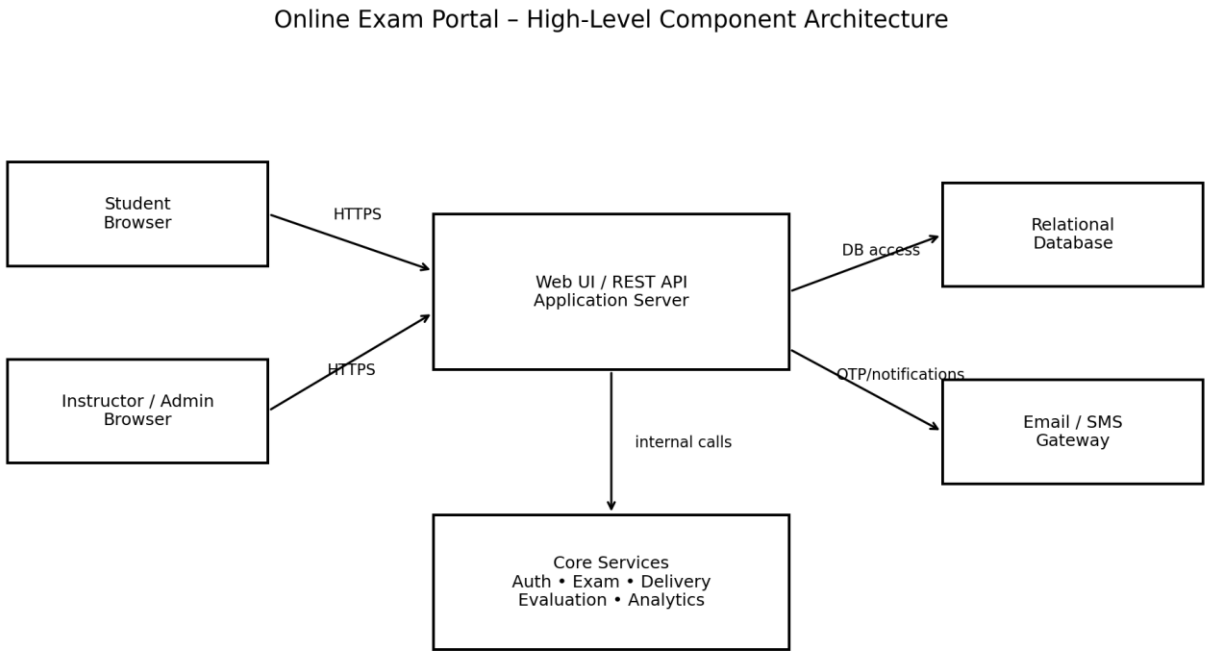
3.1 Goals & Constraints

Area	Architecture Goal / Constraint
Security	Protect credentials, exam content, answers and results; enforce TLS 1.2+ and server-side RBAC.
Reliability	Recover in-progress answers after client disconnect and maintain 99.5% availability during scheduled exam windows.
Performance	Support exam pages/question navigation within the SRS target of <=3 seconds for 90% of normal-load requests.
Scalability	Support at least 500 concurrent students taking exams without failure.
Usability	Provide responsive role-specific dashboards, keyboard navigation, high-contrast mode and a clear timed-exam interface.
Technology constraint	Use a browser client, application/API server and relational database; optional email/SMS gateway for OTP and notifications.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concern
Students	Reliable exam delivery, accurate timer, answer persistence, result privacy.
Instructors	Exam/question management, grading, result publication, analytics.
Administrators	User/role management, configuration, security and auditability.
Developers	Modularity, maintainability, clear APIs and traceability.
QA Engineers	Testable requirements, deterministic interfaces and measurable NFRs.
System/Security Administrators	TLS, credential protection, RBAC, audit logs and availability.

3.3 Component (UML) Diagram



The architecture separates browser-facing presentation from the application/API layer, core business services, persistence, and external notification integration. This structure maps directly to the major modules and interfaces specified in the SRS.

3.4 Component Descriptions

Component	Responsibilities	Key Requirements
Web UI / Browser	Role-specific dashboards; exam timer; question navigation; mark-for-review; accessibility features.	OEP-F-009..012; OEP-NF-005
Authentication & User/Role Service	Registration, login/session creation, password reset flow, role authorization.	OEP-F-001..004; OEP-SR-002..003
Exam & Question-Bank Service	Exam CRUD, question-bank management, schedule/duration/marks, question randomization.	OEP-F-005..008
Exam Delivery Service	Available-exam listing, exam session/timer enforcement, auto-save, navigation and submission.	OEP-F-009..012; OEP-NF-001..003
Evaluation & Analytics Service	Objective grading, subjective grading support, result publication, analytics and reports.	OEP-F-013..016
Relational Database	Persistent users, roles, exams, questions, attempts, answers, grades, results and audit records.	Supports all functional modules
Email/SMS Gateway Adapter	OTP/password-reset and notification integration when enabled.	OEP-F-003; SRS §3.3

3.5 Chosen Architecture Pattern and Rationale

A layered modular architecture is selected for the proposed design. The presentation layer handles browser interaction; the application/service layer implements authentication, exam, delivery and evaluation rules; the persistence layer handles relational data; and an integration adapter isolates email/SMS dependencies. This choice supports the SRS requirement for modularity, secure server-side enforcement, reliable persistence, and maintainable APIs without introducing unnecessary distributed-service complexity.

3.6 Technology Stack & Data Stores

- Client: modern web browser such as Chrome, Edge or Firefox.
- Application: web application/API server exposing REST APIs with JSON over HTTPS.
- Server environment: Linux-based application hosting, consistent with the SRS operating environment.
- Database: relational database such as MySQL or PostgreSQL, as permitted by the SRS.
- Transport security: HTTPS with TLS 1.2 or higher.
- Notification integration: email/SMS gateway API for OTP and notifications, when enabled.

3.7 Risks & Mitigations

Risk	Impact	Mitigation / Design Response
Network/client disconnect during exam	Loss of answers or incomplete submission.	Periodic server-side auto-save; restore latest persisted answers on reconnect.
Concurrent exam load exceeds capacity	Slow responses or failed submissions.	Capacity/load testing against the 500-concurrent-student requirement; stateless API sessions where practical.
Unauthorized access to exam content	Confidentiality/integrity breach.	Server-side RBAC; schedule checks; never send correct answers to the client during an active exam.
Credential compromise	Account takeover.	Salted strong password hashing; TLS; secure session handling; OTP-based password reset.
Incorrect timer behavior	Unfair exam duration.	Server-side time enforcement using the configured exam schedule and duration.

Database/service failure	Unavailable exams or results.	Backups, health monitoring, transaction-safe persistence, and recovery procedures.

3.8 Traceability to Requirements

Architecture Element	SRS Requirements
Authentication & User/Role Service	OEP-F-001..004; OEP-SR-002..003; OEP-NF-004
Exam & Question-Bank Service	OEP-F-005..008; OEP-SR-004
Exam Delivery Service	OEP-F-009..012; OEP-NF-001..003
Evaluation & Analytics Service	OEP-F-013..016
Security controls / audit	OEP-SR-001..005
Responsive/accessibility UI	OEP-NF-005

3.9 Security Architecture

The security architecture applies defense-in-depth around authentication, authorization, transport, exam-content confidentiality, and auditing. TLS 1.2+ protects network traffic. Passwords are stored only as salted strong hashes. RBAC is enforced server-side on protected endpoints. Exam questions remain unavailable before the scheduled start, and correct answers are excluded from client payloads during the exam. Login, submission, and grading events are recorded in an audit log.

3.10 Threat Model (STRIDE)

Threat	Example in OEP	Primary Control
Spoofing	Attacker attempts to impersonate a student/instructor.	Credential authentication, secure sessions, OTP reset, RBAC.
Tampering	Modification of answers, grades or exam data.	Server-side validation, authorization, protected transport, audit logs.
Repudiation	User denies performing a submission or grading action.	Audit logs for login, submission and grading.
Information Disclosure	Exposure of credentials, exam content or correct answers.	TLS, password hashing, access controls, no correct answers in exam payloads.
Denial of Service	Large concurrent load or repeated requests disrupt exam delivery.	Capacity testing, rate controls where applicable, monitoring and recovery.
Elevation of Privilege	Student attempts to call instructor/admin functionality.	Server-side RBAC on every protected API endpoint.

4. Design

4.1 Design Overview

The detailed design follows the four functional modules in the SRS: Authentication & User/Role Management, Exam & Question-Bank Management, Exam Delivery/Test-Taking Engine, and Evaluation/Results/Analytics. Shared services provide persistence, secure communication, notification integration, and auditing.

4.1.1 Logical Module Boundaries

Module	Main Responsibilities	Primary Data
M1 – Authentication & User/Role Management	Registration, login, reset, roles and sessions.	User, Role, Session/Token, OTP/Reset Request
M2 – Exam & Question-Bank Management	Exam lifecycle, questions, schedule, marks, randomization.	Exam, Question, Option, Schedule
M3 – Exam Delivery / Test-Taking Engine	Eligibility, timer, question navigation, auto-save and submission.	Exam Attempt, Answer, Review Flag
M4 – Evaluation, Results & Analytics	Auto-grade, manual grade, publication, analytics.	Grade, Result, Analytics Snapshot/Query

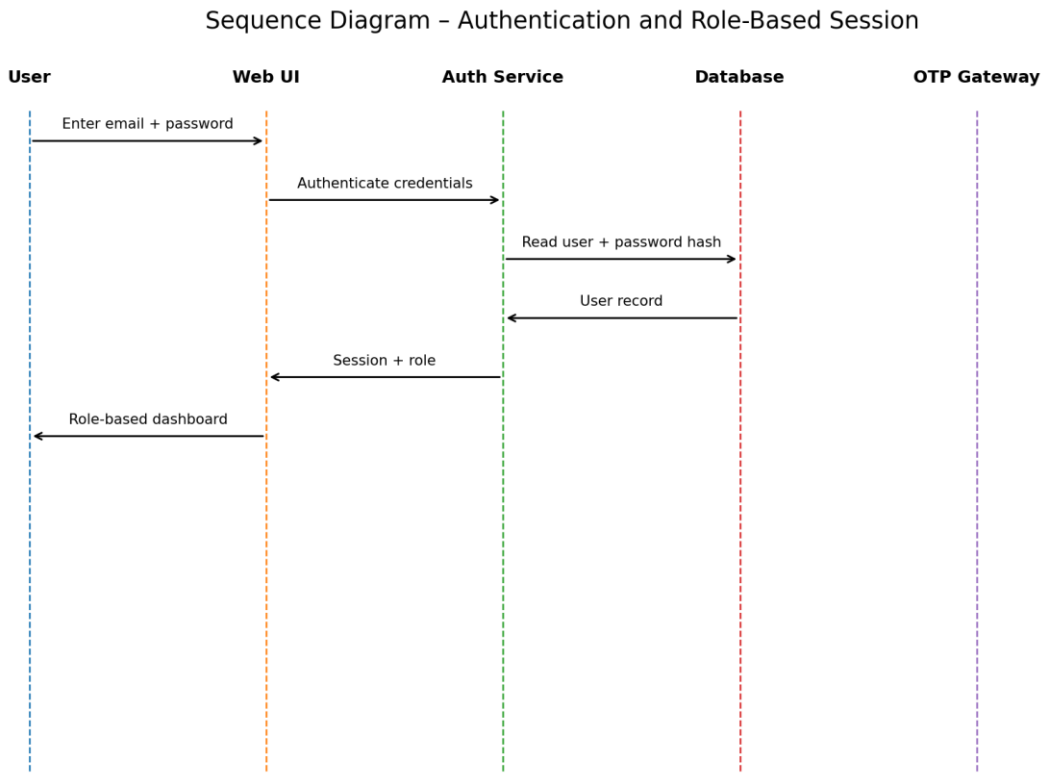
4.1.2 Core Data Model (Design-Level)

The following design-level entities are consistent with the SRS persistence needs. Exact database column names and normalization are implementation details to be finalized during development.

Entity	Important Relationships / Data
User	Has one or more roles as permitted; owns attempts and audit events.
Role	Student, Instructor or Admin; used by server-side authorization.
Exam	Created by an instructor; has schedule, duration and total marks.
Question	Belongs to the question bank; typed as MCQ, True/False or short answer.
ExamAttempt	Associates a student with an exam and tracks lifecycle/submission state.
Answer	Stores a student's response for a question and supports periodic auto-save.
Grade / Result	Stores objective score, manual marks where applicable, publication state and scorecard.
AuditLog	Records login, submission and grading actions without sensitive credential data.

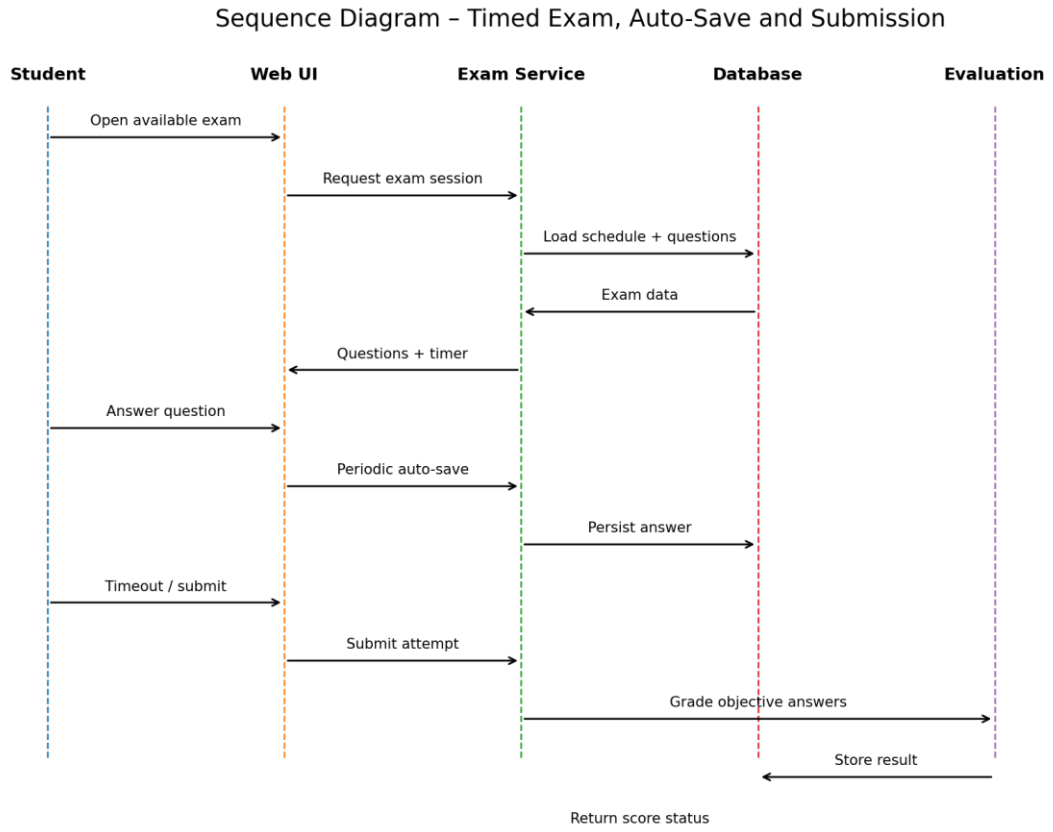
4.2 UML Sequence Diagrams

The following sequences cover two representative project flows required by the template.



Authentication flow: the browser sends credentials to the application; the authentication service validates the stored password hash and user record, creates a secure role-aware session, and returns the appropriate dashboard. Password reset can use the notification gateway for OTP/link delivery.

4.2 UML Sequence Diagram – Exam Flow



Exam flow: the student requests an eligible exam, the server validates schedule/eligibility, questions are loaded without correct answers, and the timer is enforced server-side. Answers are periodically persisted; at timeout or explicit submission, the attempt is finalized and objective questions can be graded automatically.

4.3 API Design

The SRS specifies REST APIs using JSON over HTTPS. The following design-level interfaces illustrate the required separation between client, application services and persistence.

Endpoint	Method	Request	Response	Errors / Security
/api/auth/login	POST	{ email, password }	{ status, session/token, role }	401 invalid credentials; TLS; authentication required for subsequent protected calls.
/api/auth/register	POST	{ name, email, password, roleRequest }	{ status, userId }	400 invalid data; 409 duplicate email; password hashed before persistence.
/api/exams/available	GET	Authenticated session	{ exams:[...] }	403 if unauthorized; server filters by eligibility and schedule.
/api/exams/{examId}/attempt	POST	Authenticated student session	{ attemptId, questions, timer }	403/404 when not eligible; correct answers omitted.
/api/attempts/{attemptId}/answers	PUT	{ questionId, answer }	{ status, savedAt }	401/403 invalid session/ownership; periodic auto-save.
/api/attempts/{attemptId}/submit	POST	{ attemptId }	{ status, submittedAt }	409 if already submitted; server

				enforces timeout.
/api/results/{examId}	GET	Authenticated student/instructor	{ scorecard/report }	403 if role/ownership disallows access.
/api/analytics/{examId}	GET	Instructor session	{ average, passPercent, itemAnalysis }	403 for non-instructor access.

4.3.1 API Security Rules

- All API calls use HTTPS/TLS 1.2+.
- Protected endpoints require a valid authenticated session/token.
- Authorization is checked server-side for every protected endpoint.
- Student-facing exam payloads do not include correct answers.
- Input validation is applied before database operations.
- Sensitive values such as passwords and OTP secrets are never written to logs.

4.4 Error Handling, Logging & Monitoring

4.4.1 Error Handling

- Use consistent JSON error responses containing a safe error code/message and correlation/request identifier where implemented.
- Do not reveal password hashes, correct answers, internal stack traces or database details to users.
- Use appropriate HTTP status classes: 400 for invalid requests, 401 for authentication failure, 403 for authorization failure, 404 for missing resources, and 409 for conflicting state such as repeated submission.
- When an exam client disconnects, recover the latest persisted answers rather than creating a new attempt automatically.

4.4.2 Logging & Audit

- Audit login, submission and grading actions as required by OEP-SR-005.
- Application logs should capture operational failures, latency and service errors without sensitive credentials or answer keys.
- Audit records should be access-controlled and retained according to the project's operational policy.

4.4.3 Monitoring

- Track request latency against the <=3 second target for 90% of normal-load exam/navigation requests.
- Track concurrent student sessions and load-test capacity toward the 500-student requirement.
- Track availability during scheduled exam windows.
- Track auto-save failures, submission failures, database errors and reconnect/recovery events.

4.5 UX Design

- Student dashboard: available/scheduled exams and published results.
- Instructor dashboard: exam creation, question-bank management, grading and analytics.
- Admin dashboard: user, role and system-management functions.
- Exam screen: visible countdown timer, question navigation, mark-for-review and clear submission state.
- Accessibility: keyboard navigation, high-contrast mode and responsive layout consistent with the SRS WCAG 2.1 AA target where applicable.

4.6 Open Issues & Next Steps

- Finalize the exact relational schema and indexes during implementation.
- Select the concrete application framework and deployment topology from the technology options permitted by the SRS.
- Define the exact auto-save interval and reconnection policy through implementation and testing.
- Finalize email/SMS provider and OTP expiry/rate-limit parameters.

- Execute performance testing for 500 concurrent students and verify the 90th-percentile response-time target.
- Complete security testing for TLS, password hashing, server-side RBAC, answer-key protection and audit logging.
- Validate WCAG 2.1 AA and responsive behavior across supported browsers/devices.

5. Appendices

5.1 Glossary

Term	Definition
API	Application Programming Interface used for communication between software components.
Audit Log	Persistent record of security-relevant actions such as login, submission and grading.
RBAC	Role-Based Access Control used to restrict features by Student, Instructor and Admin roles.
REST	HTTP-based architectural style used by the application's JSON APIs.
TLS	Transport security protocol used to protect client-server traffic.
OTP	One-Time Password used in password-reset verification.
Attempt	A student's active or submitted instance of an exam.

5.2 References

- Online Exam Portal – Software Requirements Specification, Team 9, Version 1.1, 09-09-2026.
- Requirements Traceability Matrix and UML Use-Case Diagrams contained in the final SRS.
- IEEE 42010 – Architecture Description guidance, as identified by the supplied SAD template.
- OWASP security guidance and NIST SP 800-160, as identified by the supplied SAD template.

5.3 Tools

- Microsoft Word / python-docx for document preparation.
- UML-style diagrams prepared for the architecture and sequence flows.
- PlantUML/draw.io may be used by the team for future refinement of formal UML artifacts.
- Swagger/OpenAPI may be used to maintain and validate the REST API contract during implementation.

5.4 SRS-to-SAD Coverage Summary

SRS Area	Covered in SAD
Authentication & User/Role Management	Architecture component, module boundary, API design, sequence flow, security controls.
Exam & Question-Bank Management	Architecture component, data model, traceability, API boundary.
Exam Delivery / Test-Taking Engine	Architecture component, exam sequence, auto-save/timer design, reliability controls.
Evaluation, Results & Analytics	Architecture component, data model, API and traceability.
NFRs	Performance, scalability, reliability, security and accessibility design responses.
Security Requirements	TLS, hashing, RBAC, answer protection and audit logging.